Presentation <1>

Link to presentation PDF and code:
https://github.com/arvindkrishna87/Team4_TheAnalysts/tree/main/Presentation1

What did each of the team members do?

Note that multiple team members can work together on the same task. In that case, you may write the same task for all of them.

Hortense (Consultant): Reviewed the model's class-weighting approach for handling the benign/high-grade class imbalance. Verified the actual class distribution (37 benign vs. 64 high-grade cases, ≈1.73:1 ratio) against the class weight currently used in training (`[2.5285, 1.0]`, selected via Optuna hyperparameter search to minimize validation loss). Found the current weight over-corrects for the imbalance by roughly 46% relative to the data's actual ratio. Proposed replacing it with a data-derived value (`[1.73, 1.0]`, computed directly as `n_high_grade / n_benign`) in `config.py`. Also produced a non-technical and a technical walkthrough of the full data pipeline (patch slicing → feature embedding → case/fold splitting → attention model) for the presentation.

Zaharaa (Consultant): Reviewed how image patches are prepared before they enter KimiaNet. Found that `precompute_pooled_features.py` feeds every patch into KimiaNet at its original size, and only drops patches smaller than 32 px (MIN_PATCH_SIZE, line 32). Because KimiaNet (DenseNet-121) shrinks each image by 32×, a patch under 64 px is summarized from a single 1×1 cell, while the 1000 × 1000 px patches KimiaNet was trained on give about 961 cells. Wrote a patch-size audit script (Code1_PatchSizeAudit_22Oct_Zaharaa.py) that measured all 81,627 patches used by the model. It found that 23.3% are under 128 px, 4.8% (about 3,900) become a single cell, and the median patch gives only 36 cells. Used the team's Case 5 attention output as an example where the most-attended patches were mostly tiny. Proposed raising MIN_PATCH_SIZE from 32 to 128 px, which keeps 76.7% of patches and all 99 cases, with a safeguard that keeps each slice's 5 largest patches if none reach 128 px. Also designed a test plan: rerun with the same seed and split as a baseline (seed 390), changing only this setting.

[Haowen, Maya, Dina] (Analyst): Analyzed the model's attention mechanism and how attention weights are mapped back to the original tissue images. Created detailed flowcharts explaining attention_analysis.py, including identification of high and low attention patches, calculation of effective attention across patches, slices, stains, and reconstruction of PND images from .pt feature files. Identified a potential indexing inconsistency between alphabetical patch ordering used in data_utils.py and the numerical ordering reconstructed in attention_analysis.py, which could cause attention weights to be associated with incorrect tissue images. Executed the model on an 18-case test set, achieving 94.4% accuracy (11 correctly classified high-grade, 6 correctly classified benign, and 1 incorrectly classified benign). Evaluated the attention visualization approach and proposed expanding stain-level comparisons, examining attention concentration, and investigating why certain highly attended patches appear blurry or pixelated.

Issues faced that haven't been resolved:

Hortense: The proposed class-weight value is derived mathematically from the verified class counts, but has not yet been empirically validated by retraining the model with the new weight, so its actual effect on performance is not yet confirmed.

Zaharaa: The proposed 128 px cutoff has not yet been tested by re-running the embeddings and retraining, so its effect on accuracy and recall is not confirmed. The link between patch size and attention is shown for one case (Case 5) only; I have not yet measured whether top-attention patches are smaller than average across all cases. Small patches may also carry real diagnostic signal (thin epithelium near the base), so the cutoff could hurt benign recall. In that case, a 64 px cutoff, which removes only 5.1% of patches, is the fallback.

Haowen, Maya, Dina: Identified a potential indexing inconsistency in attention_analysis.py: the visualization code reconstructs patch indices using numerical filename sorting, while data_utils.py organizes the model's input patches alphabetically. Additional complications include the 800-patch-per-slice limit and the possibility of missing embeddings or dropped slices, which could further misalign attention weights with their original PNG images. The actual frequency and impact of these potential mapping errors have not yet been empirically verified. Additionally, the current visualization code does not generate dedicated stain-level high- and low-attention patch comparisons, and the selected test split contained no incorrectly classified high-grade cases, preventing visualization of all four required classification outcomes. The causes of blurry high-attention patches and the degree of attention concentration across patches, slices, and stains also remain to be investigated.

References

Whose work did you build up on?

Hortense: Built on the team's existing hierarchical attention MIL framework and training configuration (`config.py`, `main.py`) developed in prior quarters of this project, including the Optuna hyperparameter search results already in place.

Zaharaa: Built on the team's existing patching and feature-extraction pipeline (precompute_pooled_features.py, attention_analysis.py) and the analysts' Case 5 attention output. Used the KimiaNet paper for the patch size and magnification it was trained on: Riasatian, A., et al. (2021). Fine-tuning and training of DenseNet for histopathology image representation using TCGA diagnostic slides. *Medical Image Analysis*, 70, 102032.

Haowen, Maya, Dina: Built on the existing framework developed by previous STAT 390 teams, particularly attention extraction and visualization functions in attention_analysis.py. Used data_utils.py, dataset.py, precompute_pooled_features.py, main.py, and config.py to understand the model's preprocessing, indexing, and evaluation pipeline and to investigate potential inconsistencies in

patch-to-image mapping. Used the model's generated attention outputs and predictions to analyze correctly and incorrectly classified cases.